When a Ruby program terminates, all threads are killed, regardless of their states. Since we don't control the scheduling of any internally created threads, it'd be easy for this program to reach the end while one or more threads are still waiting for responses. The program would terminate and we'd never get the response.

You can wait for a particular thread to finish by calling that thread's join method. The thread in which the join method is called—in our case, that's the original program—will block until the thread receiving the join call has finished. The sub thread has been off on its own, but rejoins the parent thread so the parent thread can move forward.

By calling join on each of the requested threads, you can make sure that all three requests have completed before you terminate the main program—we can see that in our code because the "we're done here" print statement will always happen after all three threads complete. Even though the join methods are called one at a time, the order and speed of the sub thread execution ultimately doesn't matter because the main thread will wait on all of them.

The join method normally returns the thread itself. If you don't want to block forever, you can give join a timeout parameter—if the timeout expires before the thread terminates, the join call returns nil. The expiration of the timeout does not actually terminate the thread, but it does allow the calling thread to continue, which might mean that the program will end before the thread is complete. Another variant of join, the method value, returns the value of the last statement executed by the thread. The value method does not have a timeout parameter.

In addition to join, a few other handy routines are used to manipulate threads. The current thread is always accessible using Thread.current. You can obtain a list of all threads using Thread.list, which returns a list of all Thread objects that are runnable or stopped. You can stop a thread with exit which is aliased as kill and terminate.

To determine the status of a particular thread, you can use status and alive?. The status method returns "run" if the thread is executing normally, "sleep" if it has been paused or is blocked, "aborting" if it is in the process of being killed. If the thread ended normally it returns false, if the thread terminated exceptionally, it returns nil. The alive? method returns true if the status is "run" or "sleep."

You can adjust the priority of a thread using priority=. Higher-priority threads will run before lower-priority threads, though the operating system is free to ignore this setting.. We'll talk more about thread scheduling, and stopping and starting threads, in just a bit.

Thread Variables

A thread can normally access any variables that are in scope when the thread is created. Variables local to the block containing the thread code are local to the thread and are not shared. But what if you need per-thread variables that can be accessed by other threads—including the main thread? Class Thread has a facility that allows thread-local variables to be created and accessed by name. You can treat the thread object as if it were a Hash, writing to elements using []= and reading them back using []. A true thread-local variable can be accessed using Thread.thread_variable_get and Thread.thread_variable_set.

In the example that follows, each thread records the current value of the variable count in a thread-local variable with the key mycount. To do this, the code uses the symbol :mycount when indexing thread objects: